

Today 12:23 PM

 **main.py**
Code **player.py**
Code **maze.py**
Code **game_engine.py**
Code

i have a python pygame maze runner project and theres a bug where the maze walls are visible but the player can move through the walls instead of being blocked by them so i can basically reach the exit directly without actually following the maze

i want to fix this using vibe coding

first analyze all the project files and figure out the exact cause of the wall collision detection bug dont make any changes yet just tell me which file and function is responsible for the issue and what exactly needs to be corrected

Yep — I checked **all 4 project files** (`main.py` , `player.py` , `maze.py` , and `game_engine.py`). The maze generation and drawing are fine. The bug is specifically in the **player collision logic**.

Exact cause

The responsible function is:

`Player._hits_wall()` in `player.py`

Right now it does this:

 **Python**

```
for px, py in [(rect.left, rect.top), ...]:  
    cr = py // CELL
```

```
cc = px // CELL
if cr < 0 or cr >= rows or cc < 0 or cc >= cols:
    return True
```

The important problem is: it **only** checks whether the player's corners are outside the maze boundaries. It never checks `walls[cr][cc]` at all.

So even though `self.walls` contains the actual maze structure, `_hits_wall()` completely ignores it. `</>` player

That's why this happens:

Walls are drawn → visually correct.

Player moves → collision function says "no wall hit."

Player walks straight through every wall.

How the files currently interact

`game_engine.py` correctly passes the maze's wall data into the player:

`</>` Python

```
self.player.move(keys, self.walls, ROWS, COLS)
```

`</>` game_engine

And `Player.move()` correctly calls `_hits_wall()` before accepting each movement:

`</>` Python

```
if not self._hits_wall(new_rect, walls, rows, cols):
    self.rect = new_rect
```

</> player

So the movement pipeline itself isn't the issue. The problem is that the function being asked to determine whether movement hits a wall doesn't actually inspect the wall definitions.

What needs to be corrected

`_hits_wall()` needs to use the structure generated by `maze.py`.

Your maze represents every cell as:

</> Plain text

[N, S, E, W]

where:

- `walls[r][c][0]` → North wall
- `walls[r][c][1]` → South wall
- `walls[r][c][2]` → East wall
- `walls[r][c][3]` → West wall

This is established in `generate_maze()`, and the maze generator correctly opens/closes those walls when carving the maze.

</> maze

</> maze

The drawing code also correctly uses those four values:

</> Python

```
if w[0]: ... # North
if w[1]: ... # South
if w[2]: ... # East
if w[3]: ... # West
```

</> game_engine

So we don't need to change `maze.py` , `game_engine.py` , or `main.py` .

 Diagnosis

File	Function	Status
maze.py	generate_maze()	✓ Correct
game_engine.py	draw_maze()	✓ Correct
game_engine.py	update()	✓ Correct
player.py	move()	✓ Basically correct
player.py	_hits_wall()	✗ BUG

Exact fix: modify `Player._hits_wall()` so that, in addition to checking the maze boundaries, it determines whether the proposed player rectangle crosses a cell boundary where the corresponding `N/S/E/W` wall is `True` .

I haven't changed **anything** in your project. This is purely the diagnosis.

And yes — this is a **perfect Vibe Coding fix** because the bug is isolated enough that we can give the coding agent a very precise instruction without letting it randomly rewrite the project.

okay so now fix the wall collision bug in player.py based on your analysis
make only the necessary changes and dont modify anything else in the project
keep the existing maze generation and movement system the same and make sure the player is

properly blocked by the maze walls

give me the updated code and explain briefly what you changed

Worked for 11s

Done. I changed **only** `player.py` , specifically `_hits_wall()` . Nothing in the maze generation, game engine, or movement system was modified.

Updated `player.py`

</> Python

```
import pygame
from game.maze import CELL

SPEED = 3

class Player:
    def __init__(self, r, c):
        self.r = r
        self.c = c
        x = c*CELL + CELL//2
        y = r*CELL + CELL//2
        self.rect = pygame.Rect(x-10, y-10, 20, 20)
        self.color = (60,120,220)

    def move(self, keys, walls, rows, cols):
        dx, dy = 0, 0
        if keys[pygame.K_LEFT] or keys[pygame.K_a]: dx = -SPEED
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx = SPEED
        if keys[pygame.K_UP] or keys[pygame.K_w]: dy = -SPEED
        if keys[pygame.K_DOWN] or keys[pygame.K_s]: dy = SPEED

        # Wall-aware movement (check cell boundaries)
        new_rect = self.rect.move(dx, 0)
```

```

if not self._hits_wall(new_rect, walls, rows, cols):
    self.rect = new_rect
new_rect = self.rect.move(0, dy)
if not self._hits_wall(new_rect, walls, rows, cols):
    self.rect = new_rect

```

What changed

Previously, `_hits_wall()` only checked whether the player went **outside the maze boundaries**. It never looked at the actual `walls` array.

</> player

Now it:

- Determines which maze cells the player's rectangle occupies.
- Checks the **East/West walls** when moving horizontally.
- Checks the **North/South walls** when moving vertically.
- Still blocks the player from leaving the maze.
- Leaves your existing `move()` system unchanged.
- Leaves `maze.py` and `game_engine.py` completely unchanged.

The actual maze wall representation remains exactly the same: `[N, S, E, W]` . </> maze

One important detail: because your movement is already separated into X and Y movement, this collision check works with the existing movement architecture rather than replacing it.

okay now

1st feature Add a BFS solver to my pygame maze. When H is pressed, highlight the shortest path from start to exit by drawing colored squares on each cell in the path. Toggle it off on second press.

2nd feature Add fog of war to my maze game. Draw a dark overlay over the whole maze surface, then cut out a circular transparent region around the player using `pygame.draw.circle` on a surface with `SRCALPHA`. Reveal only nearby cells.

3rd feature After the player solves the maze, save their time to a list (up to 5 entries, sorted ascending). Display the leaderboard on the win screen. Persist it in a JSON file called `leaderboard.json`.

4th feature Add a difficulty select screen before the maze generates. Show three buttons (Easy, Medium, Hard) with different COLS/ROWS values. Let the player click to select, then launch the maze with those dimensions.

add them to the code and send the updated codes

Worked for 31s

Analysis errored

Yep bro. I've integrated **all 4 features** while keeping the existing maze generator and movement architecture intact.

The changes are:

1. **BFS solver** → `H` toggles the shortest path.
2. **Fog of war** → circular visibility around the player.
3. **Leaderboard** → top 5 fastest completion times saved to `leaderboard.json`.
4. **Difficulty screen** → Easy / Medium / Hard before the maze starts.

The existing maze representation `[N, S, E, W]` stays unchanged, and the collision fix remains in `player.py`. `</>` maze

1. `player.py`

Replace your current `player.py` with:

`</>` Python

```
import pygame
from game.maze import CELL
```

```
SPEED = 3
```

```

class Player:
    def __init__(self, r, c):
        self.r = r
        self.c = c

        x = c * CELL + CELL // 2
        y = r * CELL + CELL // 2

        self.rect = pygame.Rect(x - 10, y - 10, 20, 20)
        self.color = (60, 120, 220)

    def move(self, keys, walls, rows, cols):
        dx, dy = 0, 0

        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            dx = -SPEED
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            dx = SPEED
        if keys[pygame.K_UP] or keys[pygame.K_w]:
            dy = -SPEED
        if keys[pygame.K_DOWN] or keys[pygame.K_s]:
            dy = SPEED

```

This fixes the original problem because `_hits_wall()` now actually checks the wall data instead of only checking the outer maze boundary. Your existing `move()` structure remains the same. `</>` player

2. game_engine.py

This is where the four new features are integrated:

</> Python

```
import json
import pygame
import time
from collections import deque

from game.maze import generate_maze, CELL
from game.player import Player

FPS = 60

BG = (240, 235, 220)
WALL_COLOR = (40, 40, 60)
EXIT_COLOR = (80, 200, 80)

DIFFICULTIES = {
    "Easy": (10, 8),
    "Medium": (15, 13),
    "Hard": (20, 16),
}

LEADERBOARD_FILE = "leaderboard.json"
FOG_RADIUS = 120

class GameEngine:

    def __init__(self):
        pygame.init()
```

3. main.py

No functional change is needed. Keep it as:

`</>` Python

```
from game.game_engine import GameEngine
```

```
if __name__ == "__main__":  
    GameEngine()
```

Your original `main.py` already just creates the `GameEngine` and starts it. `</>` `main`

4. `maze.py`

Do not change `maze.py`. Keep your existing file exactly as it is. The BFS uses the same `[N, S, E, W]` structure generated by `generate_maze()`, so the existing maze-generation system stays intact. `</>` `maze`

What you'll get

Before the maze:

Select Difficulty → Easy / Medium / Hard → Maze starts

During maze:

- WASD / arrow keys → movement
- H → show BFS shortest path
- H again → hide BFS path
- Fog only reveals the area around the player
- R → generate another maze at the selected difficulty

After winning:

- Completion time is saved
- `leaderboard.json` is updated
- Fastest 5 times are displayed

- Times are sorted fastest → slowest

One thing I deliberately **didn't** change: your underlying maze generation and player movement architecture. The original engine already passes the generated walls into `Player.move()` , so the new features are layered around that existing system rather than replacing it. `</>` game_engine `</>`

okay everything is working correctly now and all 4 features are working
can you do one final review of the entire project and verify that the wall collision fix and all the features are implemented correctly
also which code has implemented which task

Worked for 6s

Bro, I can do the final review, but there’s one important distinction: **the files currently attached to this chat are the original pre-feature versions**, not the updated versions you said you’re now running. For example, the attached `player.py` still has the old `_hits_wall()` that only checks maze boundaries. `</>` player

So I cannot honestly certify the current final files byte-for-byte from those attachments.

However, I reviewed the **final code I gave you in the previous turn**, and here is the feature-to-code mapping and verification.

 Task-by-task mapping

Task	File	Function / Code	Status
Wall collision fix	player.py	Player._hits_wall()	 Correct
BFS solver	game_engine.py	find_shortest_path()	 Correct
H key toggle	game_engine.py	handle_events()	 Correct
Draw BFS path	game_engine.py	draw_path()	 Correct

Task	File	Function / Code	Status
Fog of war	game_engine.py	draw_fog()	✔ Correct
Leaderboard storage	game_engine.py	load_leaderboard() , save_leaderboard() , save_time()	✔ Correct
Leaderboard display	game_engine.py	draw_leaderboard()	✔ Correct
Difficulty selection	game_engine.py	show_difficulty_screen()	✔ Correct
Difficulty dimensions	game_engine.py	DIFFICULTIES	✔ Correct
Maze generation	maze.py	generate_maze()	✔ Unchanged
Game startup	main.py	GameEngine()	✔ Essentially unchanged

1. Wall collision — ✔

The important correction is in:

```
player.py → Player._hits_wall()
```

It now checks the actual:

```
</> Plain text
```

```
walls[r][c] = [N, S, E, W]
```

from your existing maze generator. Your maze generator itself was already correctly creating and opening those walls. </> maze

So:

- East wall → blocks right movement
- West wall → blocks left movement
- North wall → blocks upward movement
- South wall → blocks downward movement
- Outer boundaries → also blocked

And `Player.move()` still uses the same X-then-Y movement architecture. That's exactly what we wanted.

2. BFS solver —

Implemented in:

`game_engine.py` → `find_shortest_path()`

It uses:

 Python

```
queue = deque([start])
```

and explores neighboring cells only when the corresponding maze wall is open.

So BFS isn't just drawing a random route — it is actually using your **generated maze structure** to find the shortest cell-by-cell path from:

 Plain text

```
(0, 0)
```

to:

</> Plain text

```
(rows - 1, cols - 1)
```

Then the path is reconstructed using the `previous` dictionary.

H toggle: `handle_events()`

</> Plain text

```
H pressed → show_path = True → calculate BFS path  
H pressed again → show_path = False
```

Rendering: `draw_path()`

It draws colored squares inside each path cell.

So Task 1 is properly separated into **calculation + input + rendering**.

3. Fog of war —

Implemented in:

`game_engine.py` → `draw_fog()`

The implementation correctly creates:

</> Python

```
pygame.Surface(..., pygame.SRCALPHA)
```

then fills it with a dark translucent overlay.

After that:

</> Python

```
pygame.draw.circle(
    fog,
    (0, 0, 0, 0),
    self.player.rect.center,
    FOG_RADIUS
)
```

cuts out the transparent circular visibility region.

Then:

</> Python

```
self.screen.blit(fog, (0, 0))
```

puts the fog over the maze.

That's exactly the implementation your task requested.

And importantly, the fog surface only covers the **maze area**, not the bottom HUD.

4. Leaderboard —

Three functions handle it:

load_leaderboard()

- Reads `leaderboard.json`
- Handles missing/invalid JSON
- Converts values to floats
- Sorts them

save_time()

- Adds the player's completion time
- Sorts ascending
- Keeps only 5

Conceptually:

 Plain text

```
new time
  ↓
append
  ↓
sort
  ↓
[:5]
  ↓
JSON
```

draw_leaderboard()

then displays:

Plain text

```
1. 12.32s
2. 14.87s
3. 18.21s
...
```

on the win screen.

And because `update()` immediately returns after `self.won = True`, the same completion time isn't repeatedly added every frame.

So Task 3 is correctly implemented.

5. Difficulty selection —

Implemented using:

Python

```
DIFFICULTIES = {
    "Easy": (10, 8),
    "Medium": (15, 13),
    "Hard": (20, 16),
}
```

and:

`show_difficulty_screen()`

The player clicks one of the three buttons.

Then `reset()` does:

`</>` Python

```
self.cols, self.rows = DIFFICULTIES[self.difficulty]
```

and generates the maze using those dimensions.

So the existing:

`</>` Python

```
generate_maze(cols, rows)
```

is still being used.

Nothing about your actual maze-generation algorithm needed to change.

ONE THING I CAUGHT IN MY PREVIOUS CODE

This is important.

In the `game_engine.py` code I sent you, I had:

`</>` Python

```
def __init__(self):  
    ...  
    self.run()
```

and your `main.py` has:

</> Python

```
engine = GameEngine()
engine.run()
```

That means `run()` can potentially be invoked **twice**.

Your original `main.py` is explicitly designed to create the engine and then call `run()` itself. </> main

So if your actual working code currently has **both** of those, I would remove:

</> Python

```
self.run()
```

from `GameEngine.__init__()` .

Keep:

</> Python

```
if __name__ == "__main__":
    engine = GameEngine()
    engine.run()
```

in `main.py` .

If your current code already works and you removed that duplicate call while testing, then you're good.

Final project architecture

Your project is now logically split like this:

</> Plain text

```
main.py
├── starts GameEngine
│   ├── Difficulty Selection
│   ├── Maze Generation
│   │   └── maze.py
│   ├── Player Movement
│   │   ├── player.py
│   │   └── Wall Collision
│   ├── BFS Solver
│   │   └── H key
│   ├── Fog of War
│   ├── Exit Detection
│   └── Win
│       └── Leaderboard
│           └── leaderboard.json
```

Bottom line

Feature-wise, yes — the implementation we built covers all 4 requested tasks correctly, and the original maze-generation system remains intact.

The only thing I **cannot verify from the currently attached files** is that your local project actually contains the updated code, because the uploaded files available to me are still the original versions. The attached `game_engine.py` , for example, still contains the original fixed `15 × 13` configuration and no BFS/fog/leaderboard/difficulty code. `</>` `game_engine`

If you upload your **current final `player.py` and `game_engine.py`** , I can do the true final audit and tell you “**yes, this exact version is clean**” or point out anything remaining before you submit/demo it.